

PRODUCT BACKLOG					SPRINT BACKLOG				
USER STORIES					BDD - SCENARIOS - ACCEPTANCE CRITERIA				
#P	#C	As a < type of user >	I want < some goal >	so that < some reason >	Given	And	When	And	Then
1	ADM	COMO Adminstrador de um Sistema de Gestão de Jogos	QUERO avaliar os jogos	PARA manter a veracidade das informações e o jogo válido.	Dado um resultado válido		Quando o jogo está acontecendo	E o resultado é anotado	Então o resultado deve ser aceito
					Dado um resultado inválido de valor <valor>	e participante <nomeParticipante>	Quando o jogo está acontecendo	E o resultado é anotado	Então o resultado não deve ser aceito
					Dado um Jogo Acontecendo		Quando existe dois resultados	E de participantes diferentes	Então O jogo é considerado valido
					Dado um jogo acontecendo	E um Participante	quando notar dois ou mais resultados deste participante		Então o segundo resultado deve ser ignorado

	LETICIA	COMO Aluno de um sistema de gamificação	QUERO ter média acima de 7,0	PARA ter o direito de realizar mais 3 cursos
	LUCAS			
	JOÃO			
	IZABELLY			

Dado um curso finalizado	E aluno	Quando curso está encerrado	E a média acima de 7,0	Então o aluno tem direito á mais 3 cursos
Dado um curso finalizado	E aluno	Quando curso está encerrado	E a média abaixo de 7,0	Então o aluno não tem direito á mais 3 cursos
dado um curso não finalizado	E aluno	Quando um curso não finalizado	E a média não foi lançada	Então o aluno não tem direito á mais 3 cursos
Dado um curso finalizado	E aluno	Quando curso está encerrado	E a média é exatamente 7,0	Então o aluno tem direito á mais 3 cursos

1	LUCAS	COMO Administrador de um sistema de gamificação	QUERO verificar os pagamentos	PARA dar acesso dos cursos aos alunos
---	--------------	---	--------------------------------------	--

1	IZABELLY	COMO Aluno de um sistema de gamificação	QUERO que meu plano seja atualizado para "Premium" ao completar 12 cursos	PARA passar a receber vouchers para projetos reais e moedas de recompensa
---	-----------------	---	--	--

1	JOÃO	COMO Adminstrador da plataforma de EAD	QUERO gerenciar alunos, cursos e recompensas	PARA garantir que as regras de progressão e benefício sejam aplicadas corretamente
---	-------------	--	---	---

Dado um pagamento	E alunos	Quando houver um pagamento de um valor mensal	E o curso está sendo disponibilizado	Então o aluno tem acesso aos cursos
-------------------	----------	---	--------------------------------------	-------------------------------------

Dado um aluno com plano "Básico"	E que já concluiu 11 cursos	Quando o aluno conclui o 12º curso	E o sistema verifica a quantidade e total de cursos concluídos	Então o plano do aluno deve ser atualizado para "Premium"
----------------------------------	------------------------------------	------------------------------------	--	---

Dado um aluno com cursos, médias e fórum registrados	E moedas e planos de assinaturas definidas	Quando o administrador consulta o progresso do aluno		Então o sistema exibe cursos concluídos, médias e disponíveis
--	--	--	--	---

SPRINT - IMPLEMENTAÇÃO

TDD			
And	SCENARIO	EXECUTION	RESULTS (ASSERTS)
	<pre>@Test public void deveAvaliarResultadoValidoParajogo() { var jogo = new Jogo("Caça peças"); var leonardo = new Participante("Leonardo");</pre>	<pre>jogo.anota(new Resultado(leonardo, 500.0));</pre>	<pre>assertTrue(jogo.getResultados(a() > 0); }</pre>
	<pre>@Test public void deveAvaliarResultadoInvalidoParajogo() { var jogo = new Jogo("Caça peças"); var leonardo = new Participante("Leonardo");</pre>	<pre>jogo.anota(null);</pre>	<pre>assertNull(jogo.getResulta }</pre>
	<pre>@Test public void deveTerVariosResultados() { Jogo jogo = new CriadorDeJogo() .para("Cata moedas") .resultado(new Participante("Nelson"), 150.0) .resultado(new Participante("Pedro"), 200.0) .constroi();</pre>		<pre>assertEquals(2, jogo.getResu assertEquals(150 jogo.getResultados().get(0). 0.00001); assertEquals(200 jogo.getResultados().get(1). 0.00001);</pre>
	<pre>"@" Test public void naoDeveAceitarDoisResultadosDoMesmoParticipante(){ Jogo jogo = new Jogo("Caça peças"); leonardo = new Participante("Leonardo");</pre>	<pre>jogo.anota(new Resultado(leonardo, 500.0)); //deve ser ignorado jogo.anota(new Resultado(leonardo, 600.0));</pre>	<pre>assertEquals(1, jogo.getResu assertEquals(50 jogo.getResultados().get(0) 0.00001);</pre>

	<pre> "@Test public void deveAvaliarMediaValidaParaCursoBonus(){ var curso = new Curso("Curso Terminado"); var Joao = new Aluno("Joao"); } </pre>		
	<pre> "@Test public void deveAvaliarMediaInvalidaParaCursoBonus(){ var curso = new Curso("Curso Terminado"); var Joao = new Aluno("Joao"); } </pre>		
	<pre> @Test public void naoDeveTerDireitoABonusSeCursoNaoFinalizado() { var curso = new Curso("Curso Em Andamento"); var joao = new Aluno("João"); } </pre>	<pre> boolean temDireitoABonus = joao.temDireitoABonus(curso); </pre>	<pre> assertFalse(curso.isFina assertFalse(temDireito, </pre>
	<pre> @Test public void deveAvaliarMediaLimiteBonus(){ var curso = new Curso("Curso Terminado"); var joao = new Aluno("Joao"); </pre>	<pre> curso.finalizar(joao, 7.0); </pre>	<pre> assertTrue(joao.temDireitoAM </pre>

	<pre> "@Test public void deveVerificarPagamentoMensalmente() { var pagamento = new Pagamento ("Boleto"); var aluno = new Aluno("Leonardo"); </pre>		
er para projet os reais e 3 moed as	<pre> "@Test public void deveAtualizarPlanoParaPremiumComDozeCursos(){ var izabelly = new Aluno("Izabelly"); for (int i = 0; i < 12; i++) { izabelly.concluirCurso(new Curso("Curso " + i)); } </pre>	izabelly.atualizarPlano();	<pre> assertEquals(PlanoAssinatu izabelly.getPlano()); asse izabelly.getMoeda: </pre>
participaçã o no fóru m, moed as e plano atual	<pre> public void deveConsultarProgressoDoAluno() { var aluno = new Aluno("Leonardo"); aluno.registrarCurso(new Curso("Java"), 8.5); aluno.registrarCurso(new Curso("Testes"), 6.0); aluno.registrarParticipacaoForum(5); aluno.creditarMoedas(120); aluno.definirPlano(PlanoAssinatura.BASICO); var administrador = new Administrador(); </pre>	<pre> ProgressoAluno progresso = administrador.consultarProgresso(aluno); </pre>	<pre> progresso.getCursosConclu assertEquals(1, progresso.getCursosDispor assertEquals(8.5, progresso.g 0.00001); assertEquals(5, progresso.getParticipaco assertEquals(120, progresso assertEquals(PlanoAssinat </pre>

).get(0).getMetric

dos().get(0));

ltados().size());
l.0,
.getMetrica(),
l.0,
.getMetrica(),

ltados().size());
l0,
.getMetrica(),

```
Finalizado();  
ABonus;
```

```
MaisCursos(); }
```

```
ra.PREMIUM,  
rtEquals(3,  
s()); }
```

```
idos().size());  
,  
iveis().size());  
etMedia("Java"),  
  
,  
esForum());  
.getMoedas());  
ura BASICO
```

VISÃO DO PRODUTO: Para quem gosta de jogos multiusuários os Jogos de Adivinhação e Enigma tem o objetivo de entreter, "rankiar" os vencedores para trazer diversão em bares e restaurantes para vender produtos (comidas e bebidas).

FUNCIONALIDADE / USER STORIES

US1: EU COMO Adm QUERO avaliar os jogos PARA manter a veracidade das informações e o jogo válido.

Cenários de Testes da Funcionalidade com BDD e 1º Passo do TDD

Cenário 4: Avaliando resultados de um participante

@Test

```
public void naoDeveAceitarDoisResultadosDoMesmoParticipante(){
```

```
Jogo jogo = new Jogo("Caça peças");
```

Dado um Jogo Acontecendo

```
Participante leonardo = new Participante("Leonardo");
```

E um Participante válido

```
jogo.anota(new Resultado(leonardo, 500.0));
```

```
//deve ser ignorado
```

```
jogo.anota(new Resultado(leonardo, 600.0));
```

Quando anotar dois ou mais resultados deste participante

```
assertEquals(1, jogo.getResultados().size());
```

```
assertEquals(500, jogo.getResultados().get(0).getMetrica(), 0.00001);
```

Então o segundo resultado deve ser ignorado